

CRIME RECORD MANAGEMENT SYSTEM (CRMS)

A Web-Based Crime Case Management and Automated Officer Assignment Platform

Project Report

Submitted by:

Names...

Department: Computer Science / Information Science Engineering

Institution: JSS Academy of Technical Education

Academic Year: 2025-2026

Abstract

The Crime Record Management System (CRMS) is a web-based application designed to streamline the management of criminal case records, officer assignments, citizen complaints, and case access workflows for law enforcement environments. Traditional record management systems in many institutions rely on fragmented manual processes, spreadsheets, or isolated databases, resulting in delayed case handling, poor visibility, and inefficient officer coordination.

The proposed system addresses these limitations through a centralized digital platform developed using Flask, MySQL, and a responsive frontend interface. The application provides modules for case registration, officer management, automated case assignment, complaint promotion workflows, analytics visualization, authentication, and secure citizen access requests. A rule-based automated assignment algorithm dynamically distributes cases to officers based on crime severity, rank requirements, workload balancing, and seniority considerations.

The system also integrates reCAPTCHA-based validation for public forms, asynchronous email notifications, and dynamically generated PDF dossiers for approved case access requests. RESTful APIs are used to support communication between the frontend and backend components.

The project demonstrates practical implementation of backend service design, database-driven applications, asynchronous processing, and workflow automation within a law enforcement-oriented domain. While the current implementation focuses on academic and prototype-level

deployment, the architecture is scalable and can be extended with advanced authentication systems, real-time notifications, AI-assisted analytics, and cloud deployment in future versions.

1. Introduction

1.1 Background

Law enforcement agencies manage large volumes of criminal records, complaint reports, officer assignments, and investigation updates on a daily basis. In many cases, these operations still involve semi-manual workflows, disconnected databases, or non-standardized record management approaches. Such systems often create delays in information retrieval, inefficient allocation of officers, and poor transparency in public complaint handling.

Digital transformation in public administration has increased the demand for centralized, secure, and workflow-oriented management systems capable of improving operational efficiency and accountability.

The Crime Record Management System (CRMS) was developed to provide a unified platform capable of handling:

- Criminal case management
- Officer assignment and workload balancing
- Public complaint intake
- Secure case access workflows
- Case analytics and reporting
- Automated operational processes

The system combines backend automation with a modern web-based interface to simplify record handling and improve institutional efficiency.

1.2 Problem Statement

Existing approaches to crime record management suffer from several limitations:

- Manual handling of complaint records
- Uneven distribution of cases among officers
- Lack of centralized visibility into active investigations
- Delays in citizen communication

- Weak integration between public complaint portals and internal systems
- Limited analytics support for operational monitoring

These challenges reduce operational efficiency and increase administrative overhead.

1.3 Objectives

The primary objectives of the project are:

1. To design and implement a centralized crime record management platform.
 2. To automate officer assignment using workload-aware logic.
 3. To provide secure public complaint submission workflows.
 4. To enable role-based operational workflows for officers.
 5. To generate analytical insights for case monitoring.
 6. To improve accessibility and record transparency.
 7. To reduce manual administrative effort.
-

1.4 Scope of the Project

The project focuses on a prototype-level implementation suitable for academic demonstration and controlled institutional deployment.

Included Scope

- Case management
- Officer management
- Public complaint submission
- Automated assignment logic
- Analytics dashboard
- Email notifications
- PDF dossier generation
- Authentication workflows

Excluded Scope

- Biometric authentication
- Mobile applications
- Real-time GPS tracking
- Large-scale distributed deployment
- AI-based predictive policing
- Production-grade cyber forensics integration

2. Literature Review / Existing Systems

2.1 Existing Systems

Traditional crime record systems are often built using:

- Standalone desktop software
- Spreadsheet-based management
- Paper-based documentation
- Basic database portals without automation

While some enterprise-grade law enforcement systems exist, many smaller departments and educational prototypes lack:

- Dynamic officer allocation
 - Integrated public complaint workflows
 - Automated communication systems
 - Real-time analytics
 - Modular RESTful backend design
-

2.2 Limitations of Existing Systems

Existing Limitation	Impact
Manual case assignment	Uneven workload distribution
Paper-based complaint systems	Delayed processing
Lack of centralized dashboards	Reduced visibility
Weak citizen communication	Poor transparency
Limited analytics	Difficult operational tracking

2.3 Proposed System Advantages

The proposed CRMS improves upon existing systems through:

- Centralized database-driven architecture
- Automated case assignment algorithms
- Secure complaint handling workflows
- Role-aware access control
- PDF dossier generation
- Analytics dashboards

- Asynchronous email processing
 - Modern REST API design
-

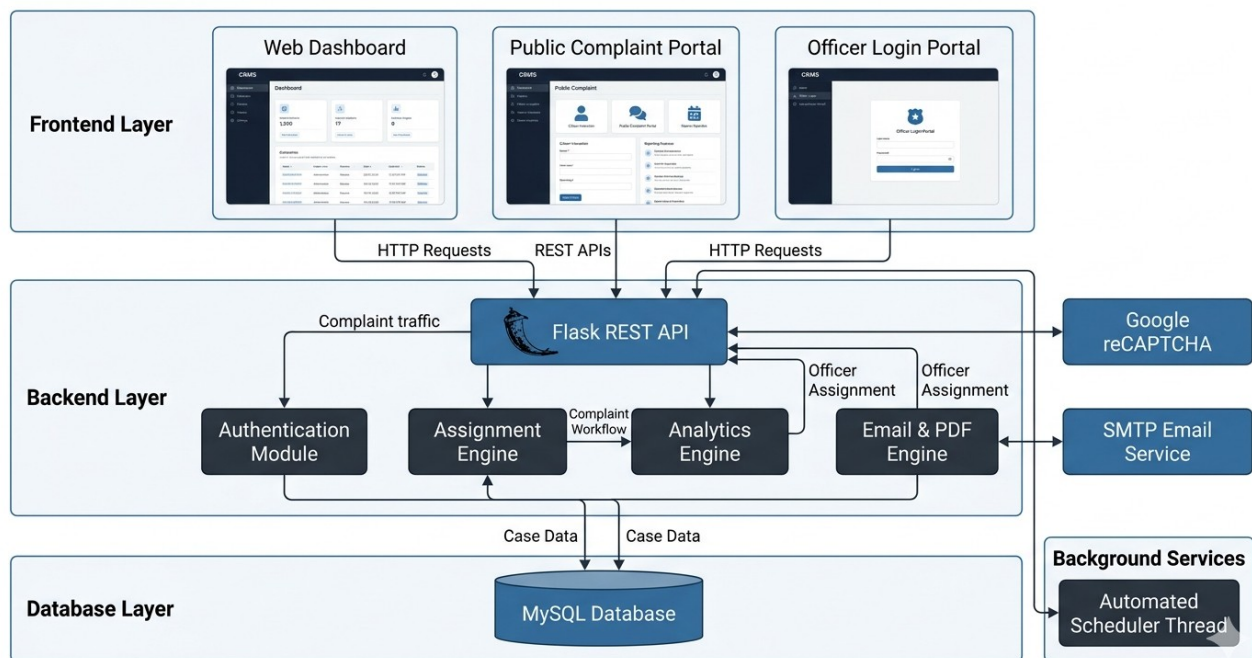
3. System Design and Architecture

3.1 Overall Architecture

The CRMS follows a layered client-server architecture.

Core Components

1. Frontend Interface
2. Flask Backend API
3. MySQL Database
4. Assignment Engine
5. Email & PDF Engine
6. Analytics Module



3.2 Technology Stack

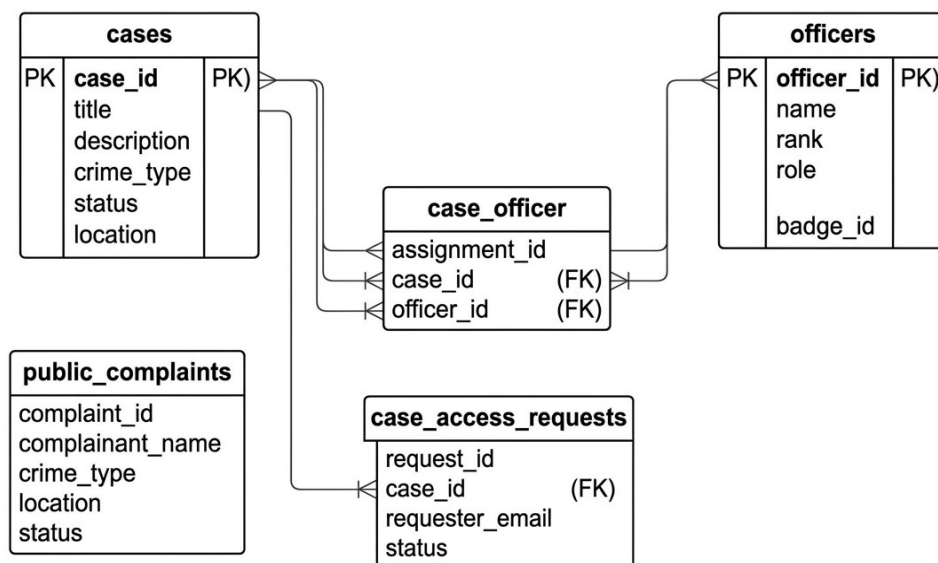
Component	Technology Used
Frontend	HTML, CSS, JavaScript
Backend	Python Flask
Database	MySQL
Authentication	bcrypt
Security	Google reCAPTCHA v2
PDF Generation	ReportLab
Email System	SMTP
Deployment Support	Procfile / Environment Variables

3.3 Database Design

The system uses a relational database architecture.

Major Tables

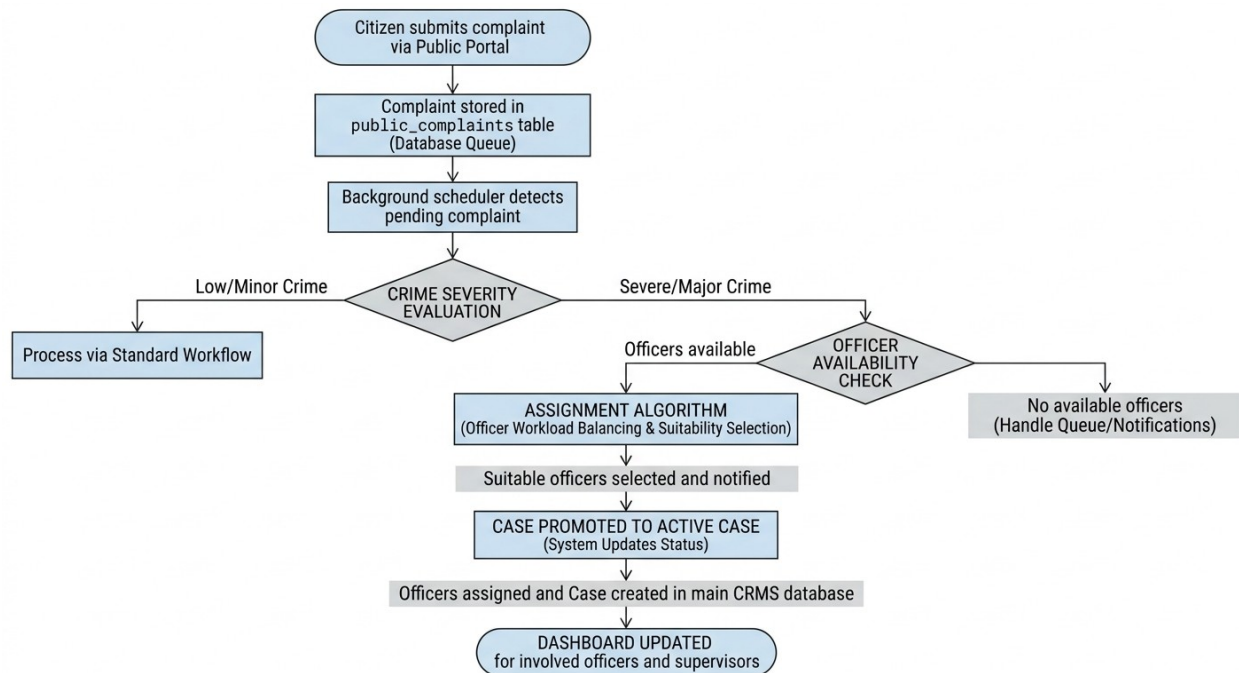
- cases
- officers
- case_officer
- public_complaints
- case_access_requests



3.4 Workflow Design

Complaint Processing Workflow

1. Citizen submits complaint
2. Complaint stored in staging table
3. Automated scheduler processes pending complaints
4. Officers selected using assignment algorithm
5. Case created and assigned
6. Officer dashboard updated



4. Implementation

4.1 Backend API Implementation

The backend was implemented using Flask with modular separation of concerns.

Major Backend Modules

Module	Responsibility
app.py	API routes and orchestration
assignment_algorithm.py	Automated officer assignment

Module	Responsibility
db_connection.py	Database pooling
email_utils.py	Email and PDF generation
queries.py	SQL abstraction layer
config.py	Environment configuration

4.2 Automated Assignment Algorithm

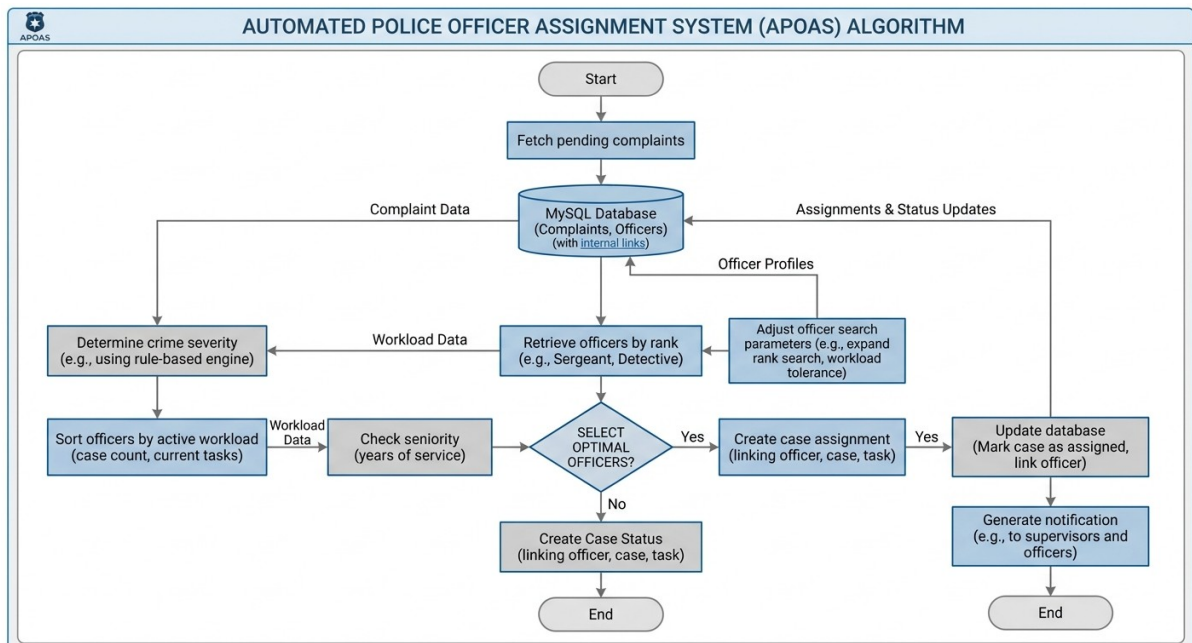
One of the major contributions of the project is the implementation of a workload-aware automated assignment system.

The algorithm:

- Classifies crimes into severity levels
- Selects officers based on rank requirements
- Balances active workload
- Uses seniority as a tiebreaker
- Automatically promotes complaints into active cases

Important Code Snippet 1

```
SEVERITY_MAP = {
    "Assault": "Critical",
    "Cyber Fraud": "High",
    "Theft": "Medium",
    "Fraud": "Medium",
    "Other": "Medium",
}
```



Explanation

This mapping converts crime categories into operational severity levels. The severity level determines the officer ranks required during automated assignment.

Important Code Snippet 2

```
ORDER BY active_cases ASC, o.join_date ASC
```

Explanation

The assignment engine prioritizes officers with lower active workloads. Seniority is used as a secondary tie-breaking factor to ensure balanced and operationally fair distribution.

4.3 REST API Design

The system exposes multiple RESTful endpoints.

Major API Categories

API Category	Purpose
/cases	Case operations
/officers	Officer management
/analytics	Dashboard analytics
/public	Public complaint workflows
/auth	Authentication
/assignments	Automated assignment management

4.4 Authentication and Security

The system integrates:

- bcrypt password hashing
- CAPTCHA validation
- Role-based access control
- Officer identity validation headers
- Secure environment variable configuration

Important Code Snippet 3

```
if not bcrypt.checkpw(password.encode(), stored_hash.encode()):  
    return _err("Invalid credentials", 401)
```

Explanation

The login system securely validates officer passwords using bcrypt hashing instead of storing plain-text credentials.

4.5 PDF and Email Engine

The system dynamically generates official case dossiers using ReportLab.

Features include:

- Styled PDF generation
- Automatic email dispatch
- Mock developer mode fallback
- Asynchronous email threads

4.6 Frontend Implementation

The frontend provides:

- Dashboard visualization
- Case browsing
- Assignment management
- Public complaint submission
- Officer management interfaces
- Analytics display

The frontend communicates with backend APIs using asynchronous HTTP requests.

4.7 Contribution Highlights

Key Contributions

- Designed modular backend architecture
- Implemented automated assignment engine
- Developed complaint promotion workflow
- Integrated reCAPTCHA security
- Implemented asynchronous email notifications
- Designed relational database schema

- Developed REST API endpoints
 - Created analytics-oriented workflows
-

5. Testing and Results

5.1 Testing Strategy

The system was tested using:

- Functional testing
 - API endpoint testing
 - Database validation
 - Authentication testing
 - Workflow testing
 - Error handling validation
-

5.2 Functional Testing

Test Case	Expected Result	Status
Officer login	Secure authentication	Passed
Complaint submission	Complaint stored successfully	Passed
Automated assignment	Officers assigned correctly	Passed
PDF generation	Valid PDF dossier generated	Passed
Email dispatch	Notification successfully triggered	Passed
Role-based access	Unauthorized access restricted	Passed

5.3 System Results

The system successfully demonstrated:

- Automated processing of public complaints
- Dynamic officer allocation
- Stable REST API communication
- Secure authentication workflows

- PDF dossier generation
- Dashboard-based analytics visualization

The prototype showed reliable performance under moderate testing conditions suitable for academic demonstration.

5.4 Screenshots of the Results

- Hero Page



- Public Complaint filing portal

The public complaint filing portal is titled 'PUBLIC SERVICES' and includes a 'BACK TO HOME' link. It features a 'FILE COMPLAINT' button and a 'REQUEST CASE ACCESS' button. The 'FILE A COMPLAINT' section includes a form with the following fields:

- COMPLAINANT NAME * (Full name)
- CONTACT NUMBER * (+91-XXXXXXXXXX)
- EMAIL ADDRESS (email@example.com)
- AADHAAR LAST 4 DIGITS * (identity verification) (XXXX)
- CRIME TYPE (Select type)
- LOCATION (Bengaluru area)
- INCIDENT DESCRIPTION (Describe the incident in detail...)

A 'SUBMIT COMPLAINT' button is located at the bottom of the form.

- Staff login

STAFF LOGIN

BENGALURU POLICE DEPARTMENT · CRMS

← BACK TO HOME

STAFF LOGIN

SECURE OFFICER ACCESS

OFFICER BADGE ID

BPD-7821

PASSWORD

SIGN IN

DEVELOPMENT CREDENTIALS

Badge: BPD-7821 · Password: crms1234

(INSPECTOR = READ+WRITE · SUB-INSPECTOR = READ ONLY)

- Staff DashBoard

BENGALURU POLICE · INTRALINK

OFFICER COMMAND DASHBOARD

← BACK TO HOME

OPERATOR: INSPECTOR ARJUN NAIR (P1)

SIGN OUT

CASE DOSSIERS

ACCESS REQUESTS

Search dossier records...

STATUS: ALL

CLASSIFICATION: ALL

BLR-019

ACTIVE

Assault - kengri

Adarsh died

ASSAULT

KENGRI

BLR-018

SOLVED

Cyber Fraud - Kengri

Some nigga called me, now my UPI died

CYBER FRAUD

KENGRI

BLR-017

CLOSED

Theft - asdsaf

afefotfads

THEFT

ASDSAF

BLR-014

ACTIVE

Cyber Fraud - Kengri

Some guy called and money gone

CYBER FRAUD

KENGRI

BLR-015

CLOSED

Assault - asdsa

ardsaas

ASSAULT

ASDSA

BLR-016

ACTIVE

Cyber Fraud - Kengri

UPI fraud

CYBER FRAUD

KENGRI

BLR-013

ACTIVE

Blah

Blah

CYBER FRAUD

BLAH

BLR-012

ACTIVE

Fraud — kengri

Blah blah

FRAUD

KENGRI

BLR-011

CLOSED

Theft — Kengri

some nigga took my bag

THEFT

KENGRI

BLR-009

ACTIVE

Data Breach - Fintech Company

Unauthorized database access exposing 2.3M user records, Cyber cell engaged. Server logs under...

CYBER FRAUD

MANYATA TECH PARK

BLR-007

CLOSED

Domestic Violence Report

Multiple domestic violence complaints filed. Protection order issued. Counseling services...

ASSAULT

HSR LAYOUT

BLR-003

ACTIVE

Assault at Commercial Street

Physical altercation between shop owners. Victim sustained head injuries. CCTV footage obtained....

ASSAULT

COMMERCIAL STREET

BLR-001

ACTIVE

BLR-004

SOLVED

BLR-008

ACTIVE

BLR-002

SOLVED

6. Challenges, Limitations and Future Work

6.1 Challenges Faced

Technical Challenges

- Managing modular backend architecture
 - Designing workload-aware assignment logic
 - Maintaining database consistency
 - Handling asynchronous email processing
 - Securing public-facing endpoints
 - Managing frontend-backend synchronization
-

6.2 Current Limitations

Although functional, the current system has several limitations:

- No JWT or token-based authentication
 - Limited production-level security hardening
 - Single-page frontend structure
 - No mobile application support
 - No real-time notification system
 - Limited scalability testing
 - Manual infrastructure deployment
-

6.3 Future Enhancements

Potential future improvements include:

- JWT-based authentication
 - Cloud deployment architecture
 - AI-assisted case prioritization
 - Real-time notification systems
 - Advanced analytics dashboards
 - Multi-department support
 - Audit logging system
 - Role hierarchy expansion
 - Mobile application integration
 - GIS and location intelligence support
-

7. Conclusion

The Crime Record Management System (CRMS) demonstrates the practical implementation of a modern web-based administrative platform for law enforcement workflows. The project successfully integrates case management, officer allocation, public complaint processing, authentication, analytics, and automated operational workflows within a unified architecture.

A major contribution of the system is the automated assignment engine, which introduces workload-aware and rank-aware distribution of cases. The integration of asynchronous email notifications, PDF dossier generation, and public complaint workflows further enhances the usability and realism of the application.

The project also highlights the importance of modular backend architecture, RESTful API design, database-driven applications, and security-aware development practices.

While the current implementation remains an academic prototype, the system establishes a strong foundation for future expansion into enterprise-scale law enforcement management platforms.

8. References

1. Flask Documentation - <https://flask.palletsprojects.com/>
 2. MySQL Documentation - <https://dev.mysql.com/doc/>
 3. ReportLab PDF Toolkit Documentation - <https://www.reportlab.com/docs/>
 4. bcrypt Python Library Documentation - <https://pypi.org/project/bcrypt/>
 5. Google reCAPTCHA Documentation - <https://developers.google.com/recaptcha>
 6. REST API Design Principles - Richardson, Leonard & Ruby, Sam
 7. Software Engineering - Ian Sommerville
 8. Database System Concepts - Silberschatz, Korth & Sudarshan
 9. Python Official Documentation - <https://docs.python.org/3/>
-

Appendix

A. Suggested Formatting Guidelines

- Font: Times New Roman
 - Font Size: 12 pt
 - Line Spacing: 1.5
 - Heading Size: 14–16 pt
 - Margin: 1 inch
-

B. Suggested Additional Diagrams

1. Use Case Diagram
 2. Sequence Diagram for Complaint Workflow
 3. Deployment Diagram
 4. Database ER Diagram
 5. Assignment Algorithm Flowchart
-

C. Repository Structure

Backend/
 app.py
 assignment_algorithm.py
 config.py
 db_connection.py
 email_utils.py
 queries.py

Frontend/
 crms_frontend.html